

variables x tipo de binario

una variable es un conjunto de datos almacenados en la memoria de la computadora para almacenar un valor que puede cambiar durante la ejecución del programa en Python no es necesario declarar la variable antes de usarla tipo de variable
La variable la variable se crean en el momento en el que se le asigna un valor con el operador = Existen estas estrategias para nombrarlas deben comenzar con una letra o un signo bajo no pueden tener espacios ni características especiales y Python distingue entre mayúsculas y minúsculas aunque no se pueden usar palabras reservadas del sistema (com if, while, def).

Tipos de datos básicos

Los datos en Python se clasifican principalmente en cuatro tipos fundamentales los enteros (int) representan números sin parte fraccionaria y pueden escribirse en formato decimal, binario, octal o hexadecimal. Los números de punto flotante (float) representan números con decimales y soportan notación científica para cifras muy grandes o pequeñas (e). Las cadenas de carácter (str) son secuencias de texto encerradas entre comillas simples o dobles por último los valores booleanos (bool) representan los valores de verdad lógica: True (verdadero) y False (falso).

Operadores Aritméticos y Jerarquía

Python incluye operadores estándar para matemáticas: Suma (+), resta (-), multiplicación (*), y división (/). Para obtener solo la parte entera de un resultado, se usa la división entera (//), y para obtener el residuo la división se utiliza (%). La exponenciación se realiza con los asteriscos (**). Python sigue una jerarquía estricta de evaluación (parentesis, exponenciación, multiplicación/división, suma/resta) y evalúa los expresiones de izquierda a derecha, excepto la exponenciación que tiene prioridad al derecho.

Entrada y Salida

La función `print()` muestra datos en la consola y permite modificar su comportamiento usando los argumentos `sep` (separar) y `end` (final de línea). Por otro lado, la función `input()` captura datos ingresados por el usuario. La diferencia entre `input()` y `raw_input()` es que `raw_input()` devuelve una cadena de texto (str) sin importar si el usuario escribió un número. Para realizar cálculos matemáticos con datos de entrada es obligatorio usar la función de conversión (casting): `int()` convierte una cadena a entero y `float()` a flotante. Con los operadores y funciones como concatenación de textos y el operador * funciona como replicación (repite el texto n veces).

Python
INSTITUTE

« PE1 -- Module 2 Quiz »

Progreso (100%)

Tu puntaje: 10/10

100%

Congratulations, you've passed the quiz!

ANÁLISIS DE LA SECCIÓN

PE1 -- Module 2 Quiz100%

Volver a presentar la prueba

Revisar prueba

R

Ricardo Daniel

1225100759.rdi@gmail.com

Iniciar sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

NUOVO

Python
INSTITUTE

« FP1: Examen Módulo 2 »

Progreso (100%)

Tu puntaje: 19/20

95%

Congratulations, you've passed the test!

ANÁLISIS DE LA SECCIÓN

PE1 -- Module 2 Test95%

Volver a presentar la prueba

Revisar prueba

R

Ricardo Daniel

1225100759.rdi@gmail.com

Iniciar sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

NUOVO

Python

INSTITUTE

MODULE (12%)

SECTION (13%)

2.1.1.6 LABORATORIO: La función print()

LABORATORIO

Tiempo Estimado

5-10 minutos

Nivel de Dificultad

Muy fácil

Objetivos

- Familiarizarse con la función `print()` y sus capacidades de formato.
- Experimentar con el código de Python.

Escenario

El comando `print()`, el cual es una de las directivas más sencillas de Python, simplemente imprime una línea de texto en la pantalla.

En tu primer laboratorio:

- Utiliza la función `print()` para imprimir la línea `"Hola, Mundo!"` en la pantalla.
- Una vez hecho esto, utiliza la función `print()` nuevamente, pero esta vez imprime tu nombre.
- Elimina las comillas dobles y ejecuta el código. Observa la reacción de Python. ¿Qué tipo de error se produce?
- Luego, elimina los paréntesis, vuelve a poner las comillas dobles y vuelve a ejecutar el código. ¿Qué tipo de error se produce esta vez?
- Experimenta tanto como puedas. Cambia las comillas dobles a comillas simples, utiliza múltiples funciones `print()` en la misma línea y luego en líneas diferentes. Observa que es lo que ocurre.

1

2

3

```
print('Ricardo Daniel')
print Ricardo Daniel
```

Console

```
>_
Hola Mundo!
File "main.py", line 1
print(Ricardo Daniel)
^
SyntaxError: invalid syntax
File "main.py", line 1
print"Ricardo Daniel"
^
SyntaxError: invalid syntax
File "main.py", line 1
print 'Ricardo Daniel'
^
SyntaxError: Missing parentheses in call to 'print'. Did you mean print('Ricardo Daniel')?
Ricardo Daniel
File "main.py", line 1
print 'Ricardo Daniel'
^
SyntaxError: Missing parentheses in call to 'print'. Did you mean print('Ricardo Daniel')?
```

Prev

Next

Ricardo Daniel

1225100759-rdi@gmail.com

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

Python

INSTITUTE

MODULE (25%)

SECTION (90%)

2.1.1.18 LABORATORIO: La función print()

LABORATORIO

Tiempo Estimado

5-10 minutos

Nivel de Dificultad

Muy fácil

Objetivos

- Familiarizarse con la función de `print()` y sus capacidades de formato.
- Experimentar con el código de Python.

Escenario

Modifica la primera línea de código en el editor, utilizando las palabras clave `sep` y `end`, para que coincida con el resultado esperado. Recuerda, utilizar dos funciones `print()`.

No cambies nada en la segunda invocación de `print()`.

Salida Esperada

Fundamentos***Programación***en...Python

1

2

3

```
print("Fundamentos", "Programación", "en", sep="***", end="...")
print("Python")
```

Console

```
>_
Fundamentos Programación en
Python
Fundamentos***Programación***en...Python
Fundamentos***Programación***en...Python
```

Prev

Next

Ricardo Daniel

1225100759-rdi@gmail.com

Iniciar sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

PythonINSTITUTE

2.4.1.9 LABORATORIO: Variables, un sencillo convertidor

MODULO (72%)

SECTION (82%)

LABORATORIO

Tiempo Estimado

10 minutos

Nivel de Dificultad

Fácil

Objetivos

- Familiarizarse con el concepto de variables y trabajar con ellas.
- Realizar operaciones básicas y conversiones.
- Experimentar con el código de Python.

Escenario

Millas y kilómetros son unidades de longitud o distancia.

Teniendo en mente que `1` milla equivale aproximadamente a `1.61` kilómetros, complementa el programa en el editor para que convierta de:

- Millas a kilómetros.
- Kilómetros a millas.

No se debe cambiar el código existente. Escribe tu código en los lugares indicados con `###`. Prueba tu programa con los datos que han sido provistos en el código fuente.

Pon mucha atención a lo que esta ocurriendo dentro de la función `print()`. Analiza como es que se proveen múltiples argumentos para la función, y como es que se muestra el resultado.

Nota que algunos de los argumentos dentro de la función `print()` son cadenas (por ejemplo `"millas son"`), y otros son variables (por ejemplo `miles`).

```
1 kilometers = 12.25
2 miles = 7.38
3
4 miles_to_kilometers = miles * 1.61
5 kilometers_to_miles = kilometers / 1.61
6
7 print(miles, "millas son", round(miles_to_kilometers, 2), "kilómetros")
8 print(kilometers, "kilómetros son", round(kilometers_to_miles, 2), "millas")
9
```

Console >_

7.38 millas son 11.88 kilómetros
12.25 kilómetros son 7.61 millas

Ricardo Daniel1225100759.rdi@gmail.com

Iniciar sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

PythonINSTITUTE

2.4.1.10 LABORATORIO: Operadores y expresiones

MODULO (74%)

SECTION (91%)

LABORATORIO

Tiempo Estimado

10-15 minutos

Nivel de Dificultad

Fácil

Objetivos

- Familiarizarse con los conceptos de números, operadores y operaciones aritméticas en Python.
- Realizar cálculos básicos.

Escenario

Observa el código en el editor: lee un valor `floatante`, lo coloca en una variable llamada `x`, e imprime el valor de la variable llamada `y`. Tu tarea es completar el código para evaluar la siguiente expresión:

$$3x^3 - 2x^2 + 3x - 1$$

El resultado debe ser asignado a `y`.

Recuerda que la notación algebraica clásica muy seguido omite el operador de multiplicación, aquí se debe de incluir de manera explícita. Nota como se cambia el tipo de dato para asegurarnos de que `x` es del tipo `floatante`.

Mantén tu código limpio y legible, y pruébalo utilizando los datos que han sido proporcionados. No te desanimes por no lograrlo en el primer intento. Se persistente y curioso.

```
1 x = 1.0 # o cualquier otro dato de prueba que quieras ingresar en la línea 1
2 x = float(x)
3 # escribe tu código aquí
4 y = 3 * (x ** 3) - 2 * (x ** 2) + 3 * x - 1
5 print("y =", y)
6
```

Console >_

y = 3.0

Ricardo Daniel1225100759.rdi@gmail.com

Iniciar sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

PythonINSTITUTE

2.5.1.2 LABORATORIO: Comentarios

MODULO (78%)

SECTION (67%)

LABORATORIO

Tiempo Estimado

5 minutos

Nivel de Dificultad

Muy Fácil

Objetivos

- Familiarizarse con el concepto de comentarios en Python.
- Utilizar y no utilizar los comentarios.
- Reemplazar los comentarios con código.
- Experimentar con el código de Python.

Escenario

El código en el editor contiene comentarios. Intenta mejorarlo: agrega o quita comentarios donde consideres que sea apropiado (en ocasiones el remover un comentario lo hace mas legible), además, cambia el nombre de las variables donde consideres que esto mejorará la comprensión del código.

NOTA

Los comentarios son muy importantes. No solo hacen que el programa sea **más fácil de entender**, pero también sirven para **deshabilitar aquellas partes de código que no son necesarias** (por ejemplo, cuando se necesita probar cierta parte del código, e ignorar el resto). Los buenos programadores **describen** cada parte importante del código, y dan **nombres significativos** a variables, debido a que en ocasiones es mucho más sencillo dejar el comentario dentro del código mismo.

Es bueno utilizar nombres de variables **legibles**, y en ocasiones es mejor **dividir el código** en partes con nombres (por ejemplo en funciones). En algunas situaciones, es una buena idea escribir los pasos de como se realizaron los cálculos de una forma sencilla y clara.

Una cosa mas: puede ocurrir que un comentario contenga una pieza de información incorrecta o errónea.

```
1 # Calcula la cantidad de segundos que hay en un número determinado de horas.
2 HORAS_A_CALCULAR = 2
3 SEGUNDOS_POR_HORA = 3600
4
5 # Calcular y mostrar los resultados
6 segundos_totales = HORAS_A_CALCULAR * SEGUNDOS_POR_HORA
7
8 print("Horas: ", HORAS_A_CALCULAR)
9 print("Segundos en Horas: ", segundos_totales)
10 print("Adiós")
11
```

Console >_

Horas: 2
Segundos en Horas: 7200
Adiós

Ricardo Daniel1225100759.rdi@gmail.com

Iniciar sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

PythonINSTITUTE

2.6.1.9 LABORATORIO: Entradas y salidas simples

MODULE (32%)

SECTION (89%)

LABORATORIO

Tiempo Estimado

5-10 minutos

Nivel de Dificultad

Fácil

Objetivos

- Familiarizarse con la entrada y salida de datos en Python.
- Evaluar expresiones simples.

Escenario

La tarea es completar el código para evaluar y mostrar el resultado de cuatro operaciones aritméticas básicas.

El resultado debe ser mostrado en consola.

Quizá no podrás proteger el código de un usuario que intente dividir entre cero. Por ahora, no hay que preocuparse por ello.

Prueba tu código - ¡Produce los resultados esperados!

No te mostraremos ningún dato de prueba, eso sería demasiado sencillo.

```
1 # ingresa un valor flotante para la variable a aquí
2 a = float(input("Ingresa el primer número: "))
3 # ingresa un valor flotante para la variable b aquí
4 b = float(input("Ingresa el segundo número: "))
5
6 # muestra el resultado de la suma aquí
7 print("Suma:", a + b)
8 # muestra el resultado de la resta aquí
9 print("Resta:", a - b)
10 # muestra el resultado de la multiplicación aquí
11 print("Multiplicación:", a * b)
12 # muestra el resultado de la división aquí
13 print("División:", a / b)
14
15 print("\n¡Eso es todo, amigos!")
16
```

Console >_

Ingresa el primer número: 1
Ingresa el segundo número: 2
Suma: 3.0
Resta: -1.0
Multiplicación: 2.0
División: 0.5

¡Eso es todo, amigos!

Ricardo Daniel
1225100759.rdi@gmail.com

Iniciar sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

PythonINSTITUTE

2.6.1.10 LABORATORIO: Operadores y expresiones

MODULE (35%)

SECTION (77%)

LABORATORIO

Tiempo Estimado

20 minutos

Nivel de Dificultad

Intermedio

Objetivos

- Familiarizarse con los conceptos de números, operadores y expresiones aritméticas en Python.
- Comprender la precedencia y asociatividad de los operadores de Python, así como el correcto uso de los paréntesis.

Escenario

La tarea es completar el código para poder evaluar la siguiente expresión:

$$x + \frac{1}{x + \frac{1}{x + \frac{1}{x}}}$$

El resultado debe de ser asignado a `y`. Se cauteloso, observa los operadores y priorízalos. Utiliza cuantos paréntesis sean necesarios.

Puedes utilizar variables adicionales para acortar la expresión (sin embargo, no es muy necesario). Prueba tu código cuidadosamente.

```
1 x = float(input("Ingresa el valor para x: "))
2
3 # Escribe tu código aquí.
4 y = 1 / (x + 1 / (x + 1 / (x + 1 / x)))
5
6 print("y = ", y)
7
```

Console >_

Ingresa el valor para x: 1
y = 0.6000000000000001

Ricardo Daniel
1225100759.rdi@gmail.com

Iniciar sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

PythonINSTITUTE

2.6.1.11 LABORATORIO: Operadores y expresiones

MODULE (34%)

SECTION (85%)

LABORATORIO

Tiempo Estimado

15-20 minutos

Nivel de Dificultad

Fácil

Objetivos

- Mejorar la habilidad de implementar números, operadores y operaciones aritméticas en Python.
- Utilizar la función `print()` y sus capacidades de formato.
- Aprender a expresar fenómenos del día a día en términos de un lenguaje de programación.

Escenario

La tarea es preparar un código simple para evaluar o encontrar el **tiempo final** de un periodo de tiempo dado, expresándolo en horas y minutos. Las horas van de 0 a 23 y los minutos de 0 a 59. El resultado debe ser mostrado en la consola.

Por ejemplo, si el evento comienza a las **12:17** y dura **59 minutos**, terminará a las **13:16**.

No te preocupes si tu código no es perfecto, está bien si acepta una hora inválida, lo más importante es que el código produzca una salida correcta acorde a la entrada dada.

Prueba el código cuidadosamente. Pista: utilizar el operador `%` puede ser clave para el éxito.

```
1 hour = int(input("Hora de inicio (horas): "))
2 mins = int(input("Minuto de inicio (minutos): "))
3 dura = int(input("Duración del evento (minutos): "))
4
5 # Escribe tu código aquí.
6
7 # 1. Calcular el total de minutos sumando los minutos iniciales y la duración
8 total_mins = mins + dura
9
10 # 2. Calcular cuántas horas extra se generan a partir de los minutos totales
11 extra_hours = total_mins // 60
12
13 # 3. Obtener los minutos finales restantes usando el operador módulo
14 final_mins = total_mins % 60
15
16 # 4. Calcular la hora final sumando las horas iniciales y las extra,
17 # y aplicar el módulo 24 para que se mantenga en el formato de 0 a 23
18 final_hour = (hour + extra_hours) % 24
19
20 # 5. Mostrar el resultado final en la consola separando la hora y los minutos por dos puntos
21 print(final_hour, final_mins, sep=":")
22
```

Console >_

Hora de inicio (horas): 7
Minuto de inicio (minutos): 4
Duración del evento (minutos): 3
7:7

Ricardo Daniel
1225100759.rdi@gmail.com

Iniciar sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome

PythonINSTITUTE

2.6.1.12 LABORATORIO: Operadores y expresiones

MODULE (34%)

SECTION (85%)

LABORATORIO

Tiempo Estimado

15-20 minutos

Nivel de Dificultad

Fácil

Objetivos

- Mejorar la habilidad de implementar números, operadores y operaciones aritméticas en Python.
- Utilizar la función `print()` y sus capacidades de formato.
- Aprender a expresar fenómenos del día a día en términos de un lenguaje de programación.

Escenario

La tarea es preparar un código simple para evaluar o encontrar el **tiempo final** de un periodo de tiempo dado, expresándolo en horas y minutos. Las horas van de 0 a 23 y los minutos de 0 a 59. El resultado debe ser mostrado en la consola.

Por ejemplo, si el evento comienza a las **12:17** y dura **59 minutos**, terminará a las **13:16**.

No te preocupes si tu código no es perfecto, está bien si acepta una hora inválida, lo más importante es que el código produzca una salida correcta acorde a la entrada dada.

Prueba el código cuidadosamente. Pista: utilizar el operador `%` puede ser clave para el éxito.

```
1 hour = int(input("Hora de inicio (horas): "))
2 mins = int(input("Minuto de inicio (minutos): "))
3 dura = int(input("Duración del evento (minutos): "))
4
5 # Escribe tu código aquí.
6
7 # 1. Calcular el total de minutos sumando los minutos iniciales y la duración
8 total_mins = mins + dura
9
10 # 2. Calcular cuántas horas extra se generan a partir de los minutos totales
11 extra_hours = total_mins // 60
12
13 # 3. Obtener los minutos finales restantes usando el operador módulo
14 final_mins = total_mins % 60
15
16 # 4. Calcular la hora final sumando las horas iniciales y las extra,
17 # y aplicar el módulo 24 para que se mantenga en el formato de 0 a 23
18 final_hour = (hour + extra_hours) % 24
19
20 # 5. Mostrar el resultado final en la consola separando la hora y los minutos por dos puntos
21 print(final_hour, final_mins, sep=":")
22
```

Console >_

Hora de inicio (horas): 7
Minuto de inicio (minutos): 4
Duración del evento (minutos): 3
7:7

Ricardo Daniel
1225100759.rdi@gmail.com

Iniciar sesión en tu teléfono

Contraseñas y Autocompletar

Gestionar tu cuenta de Google

Personalizar perfil

La sincronización está activada

Añadir perfil de Chrome

Perfil de invitado abierto

Gestionar perfiles de Chrome